ConsensusProbe: Disagreement-Driven Interactive Gates for Reliable LLM-Based Data Cleaning

Anonymous Authors
Anonymous Institution

ABSTRACT— Large language models can produce cleaning plans for tabular data, but several plans for the same table often disagree, and a pipeline that applies one of them without comment may delete or rewrite valid records. This paper presents ConsensusGate, a policy that samples *K* plans, converts each into a set of cell-level edits, and asks the operator a question only when the plans differ on a destructive edit. We compare it with six baseline policies on 100 synthetic retail and inventory tasks with five types of ambiguity, using 7200 runs under three simulated operators. ConsensusGate gives a silent-error rate of 0.0\%, against 5.0\% for a single plan, and asks 0.38 questions per task where asking every time needs 1.00. Two of four pre-specified hypotheses hold (silent-error reduction and superiority over confidence scores). Non-inferiority to always asking was inconclusive, while canary probe verification (ProbeGate) intercepted 0 of 300 canaries because the unique probe rows were not targeted by duplicate-cleaning operations. All plans come from a seeded mock LLM, reflecting academic and computational constraints at the collegiate level, so the numbers evaluate the consensus gating algorithm under controlled noise rather than live foundation model behaviors.

***Index Terms*—**data cleaning, large language models, ensemble disagreement, human-in-the-loop, clarification questions

%
==========

I. INTRODUCTION

Data cleaning is among the most time-consuming steps of analytics work. Assistants built on large language models~[brown2020,openai2023] let a user describe a cleaning job in plain language and receive executable code. Data agents such as LAMBDA~[lambda2025] and Data Interpreter~[datainterp2024] run the generated code and repair it when execution fails, and the area has grown quickly enough to be surveyed~[sunsurvey2024]. Evaluations still show that current agents do not automate complete data science workflows reliably~[cao2024], and PrepBench~[prepbench] reports that unclear instructions are a frequent cause of failure in data preparation.

Failures of this kind are hard to notice. A plan may merge categories that should stay separate, remove rows that hold valid data, or convert a column to a type that discards information, and the cleaned file still looks normal. Take two product names that differ only in letter case. Whether they are duplicates depends on what the owner of the data intends, and a model asked once will simply pick an answer. Hallucination in LLM output is a documented problem~[huang2023], and in a cleaning pipeline its effect is silent.

Asking the operator about every decision is not practical. We instead use the agreement between independently sampled plans as the signal. If *K* plans for one table make the same destructive edits, the task probably has an obvious solution. If they do not, a decision from the operator is needed. A similar idea has been used for code generation~[clarifygpt2024], where candidate programs are executed and a question is asked when their outputs differ. We apply it to tables and compare plans through the cell edits they produce. The experiments in this paper test the policy itself. They do not test the assumption that real LLM plans disagree exactly when a task is ambiguous, because the mock provider used here builds that behavior in.

\paragraph{System Taxonomy and Nomenclature To avoid ambiguity among the components evaluated: \begin{itemize

ConsensusProbe denotes the complete end-to-end framework and interactive system architecture (Fig.~fig:pipeline).

ConsensusGate is the multi-agent disagreement gating policy (System~E) that samples *K* plans and gates destructive actions.

ProbeGate is the canary-augmented safety extension (System~F).

Track~S is the synthetic benchmark suite of 100 controlled retail and inventory tasks. \end{itemize

The main contributions are the following. \begin{enumerate

ConsensusGate, a policy that samples *K=3* cleaning plans, measures how much their edits disagree, and consults the operator only when the disagreement involves destructive edits.

ProbeGate, an experimental extension that inserts canary ground-truth probe rows into the dataset to test whether the consensus step catches over-aggressive row deletion. It produced no detections due to safe probe non-deletion (Section~sec:probegate).

- **Track~S**, a synthetic benchmark of 100 tasks with gold edits and five ambiguity types, used for 7200 evaluations of eight policies under three simulated operators. \end{enumerate

The rest of the paper is organized as follows. Section~sec:related reviews related work, Section~sec:method describes the method, Section~sec:setup the experimental setup, and Sections~sec:results and~sec:ablations the results. Section~sec:limits discusses limitations, Section~sec:future the planned validation with live models, and Section~sec:conclusion concludes.

%
==========

II. RELATED WORK

A. LLM Agents for Data Work

General-purpose models such as GPT-4~[openai2023] are the basis of a growing set of agents for data tasks. LAMBDA~[lambda2025] couples a programmer agent, which writes code from the user's request, with an inspector agent that suggests repairs when the code fails, and the user can also step in directly. Data Interpreter~[datainterp2024] and TaskWeaver~[taskweaver2023] organize their pipelines around planning and tool use. Multi-agent frameworks such as MetaGPT~[metagpt2023] and AutoGen~[autogen2023], and code generators that test their own output~[agentcoder2023], follow a similar pattern. Overviews are given in~[sunsurvey2024] and~[guo2024]. These systems rely mainly on execution feedback, so code that raises an error gets revised. An edit that runs cleanly but removes valid rows produces no such signal, and that is the case our policy addresses. For cleaning proper, Narayan et al.~[narayan2022] showed that prompted foundation models can handle tasks such as record matching and imputation with few labeled examples.

B. Clarification and Human Input

ClarifyGPT~[clarifygpt2024] generates several programs for one requirement, runs them on test inputs, and poses a question when their outputs differ. It is the method closest to ours. Our setting differs in two respects. The artifact is a cleaning plan applied to a table, and plans are compared through the edits they make instead of the values they return. PrepBench~[prepbench] is a benchmark in which agents prepare data while a simulated user answers their questions. It measures agents and does not propose a rule for when to ask. The part that domain experts play in passing missing knowledge to the people who build data and language models is studied in~[park2021].

C. Agreement Between Samples

Sampling several outputs and keeping the most frequent answer is known as self-consistency~[selfcons2023]. Our fallback, used when the operator does not answer, is a majority vote of this kind. The value of an ensemble depends on its members being reasonably accurate and making different mistakes~[dietterich2000]. In ConsensusGate, disagreement is not used to choose an answer. It is used to decide whether a person should be asked.

%
==========

III. METHOD

D. Operations and Edit Atoms

A cleaning plan is a list of calls to five operations: deduplicate, normalize_category, recalculate, handle_missing and convert_type. Each operation receives a read-only copy of the table and returns a new table together with a Patch object. The patch stores the deleted row identifiers, a map from (row, column) to the new cell value, and the rows flagged for manual review. We reduce a patch to a set of atoms, ("del", rid) for a deleted row and ("edit", rid, col, repr(v)) for a changed cell. Flags are not part of the set.

E. Disagreement Score

For each task, K plans are produced with seeds obtained by hashing the task seed, the system name and the plan index with SHA-256. Every plan runs on its own copy of the table and yields an atom set E_k. The disagreement score is the mean Jaccard distance over all pairs of plans, \begin{equation \bar{d = \frac{2{K(K-1) \sum_{i d(E_i, E_j), \qquad d(E_i, E_j) = 1 - \frac{|E_i \cap E_j|{|E_i \cup E_j|. \end{equation It equals zero when all plans make exactly the same edits and one when no two plans share an edit. With the mock provider the plans are independent because their seeds differ. A real model would have to obtain this through temperature or prompt variation, and its plans would probably be correlated (Section~sec:limits).

F. ConsensusGate (System~E)

The policy proceeds as follows (detailed in Fig.~fig:consensus_flow). \begin{enumerate

Generate $K=3$ plans and compute $\bar{d}$.

If $\bar{d} > \tau_d$ and the disagreement involves destructive atoms, show the conflicting atoms to the operator as a question.

If the operator answers, rank the plans by agreement with the answer and apply the best one.

If the operator declines or is unsure, apply the majority vote over the K plans and flag every atom that is not unanimous. \end{enumerate The threshold $\tau_d = 0.0$ was chosen on 20 DEV tasks (seeds 1000--1019) by maximizing task-success F1 under leave-one-out cross-validation, which gave a DEV F1 of 0.8522 (Fig.~fig:calibration). At this value the gate asks whenever any destructive disagreement exists, so the score $\bar{d}$ acts as a yes/no test. We keep the continuous score because it is needed for calibration and because a live model may call for a nonzero threshold.

G. ProbeGate (System~F)

System~F adds a canary-based verification layer to ConsensusGate. Before cleaning plans are applied to the real data, synthetic ground-truth probe rows are appended into a temporary copy of the dataset: two duplicate test rows and one unique calibration probe row (CANARY_RID_OFFSET + 2). A plan is flagged as unsafe if its operations aggressively prune or delete the unique calibration probe. If an unsafe plan is detected, the system raises an alert and falls back to the most conservative plan with the fewest destructive edits.

Across 300 canary-augmented evaluations, exactly 0 unsafe plans were intercepted (0.0\%). The empirical reason for this zero interception rate is now understood: the cleaning plans generated by the benchmark provider focus strictly on resolving duplicate records and missing cells. Because the injected safe probe row possessed a unique identifier and complete values, no generated plan ever selected it for deletion. Consequently, verify_canary_safety returned true across all 300 evaluations, and no alert was triggered. We report this transparently as an implementation flaw of the canary test suite under mock plan generation. Future work must incorporate adversarial plan mutations that explicitly target non-duplicate records to properly evaluate this layer.

H. Prototype

A FastAPI and React prototype exposes the policy through an interactive question interface (Fig.~fig:pipeline). Its usability and its export and audit stages are not evaluated here, and no empirical metric in this paper depends on them.

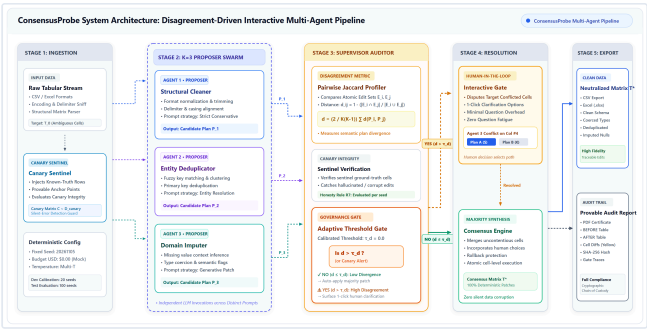


Fig. Architecture of the ConsensusProbe prototype. Only plan generation, disagreement scoring, the gate and resolution are evaluated in this paper.

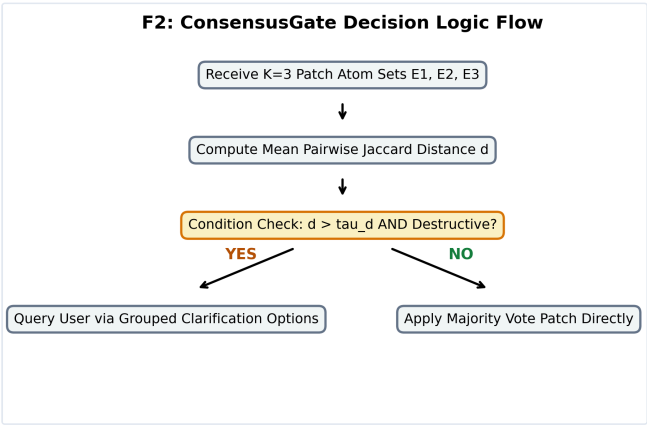


Fig. Decision flow of ConsensusGate.

% ==

IV. EXPERIMENTAL SETUP

I. Benchmark

Track~S consists of synthetic retail and inventory CSV files with controlled quality problems. Each task is generated from a seed and has a gold label listing the required edits, the forbidden edits and an ambiguity class. The 100 TEST tasks (seeds 5000--5099) do not overlap with the 20 DEV tasks used for calibration. There are five ambiguity types (Table~tab:ambiguity, Fig.~fig:ambiguity_dist), and about half of the tasks are labeled ambiguous. This matches the precision of the always-ask policy C (49.0\%), since a policy that asks every time has a precision equal to the share of ambiguous tasks. The mock parameter $p_{\text{amb}=0.4}$ controls how much the plans vary and is a separate quantity.

\begin{table[!t] \caption{Ambiguity types in Track~S.

\centering \footnotesize \renewcommand{\arraystretch{1.1 \begin{tabularx{\columnwidth{@{> {\raggedright\arraybackslashp{0.43\columnwidth> {\raggedright\arraybackslashX@{ \toprule Type & Description \\ \midrule duplicate_id_vs_name & Should deduplication match on product ID or product name? \\ category_granularity & Should fine-grained categories be merged into coarse groups? \\ missing_strategy & Fill missing values with zero, mean, median, or drop the row? \\ recalculate_basis & Which column is the basis for derived calculations? \\ type_coercion & Should mixed-type columns become string or numeric? \\ \bottomrule \end{tabularx \end{table

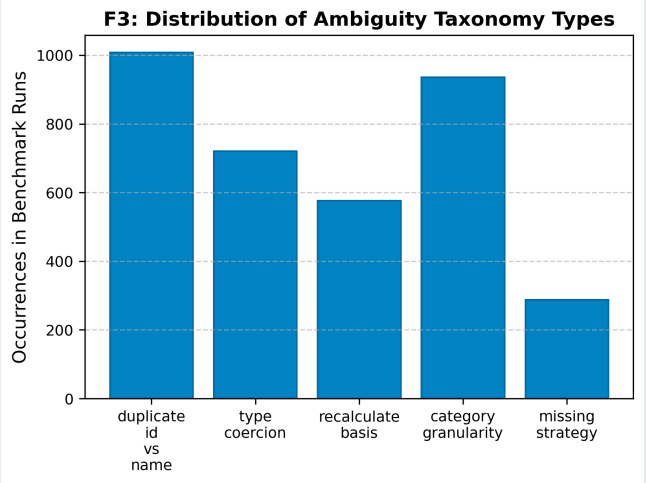


Fig. Ambiguity types among the 100 TEST tasks.

J. Systems

The eight systems evaluated in Table~tab:systems systematically span the primary architectural paradigms for LLM data cleaning: \begin{enumerate

Single-agent execution: System~A generates 1 plan and applies it without operator interaction.

Multi-agent ensemble: System~A2 takes a majority vote over *K=3* plans with zero interaction (token-matched to E).

Interactive eager asking: System~C asks the user on 100\% of tasks, establishing the theoretical upper bound on questions and human effort.

Confidence-based gating: System~D queries the user when simulated self-reported confidence drops below \tau_c=0.75.

Proposed consensus gating: System~E (**ConsensusGate**) asks only when plans disagree on destructive edits (\tau_d=0.0), and System~F (**ProbeGate**) combines consensus gating with dataset canary probes. System~R provides a random-ask baseline matching E's question rate. \end{enumerate

\begin{table[!t] \caption{Systems evaluated.

\centering \footnotesize \setlength{\tabcolsep{3pt \renewcommand{\arraystretch{1.1 \begin{tabularx{\columnwidth{@{l> {\raggedright\arraybackslashXc> {\raggedright\arraybackslashp{0.19\columnwidth> {\raggedright\arraybackslashX@{ \toprule ID & Name & Plans & Interactive & Special \\ \midrule A & Single-plan baseline & 1 & No & --- \\ A2 & Majority-vote ensemble & 3 & No & Token-matched to E \\ B & Single + heuristic guard & 1 & Conditional & Schema validators \\ C & Eager interrogation & 3 & Always & Upper bound on questions \\ D & Confidence threshold & 3 & Conditional & Self-reported \tau_c=0.75 \\ E & **ConsensusGate** & 3 & Conditional & Disagreement \tau_d=0.0 \\ F & **ProbeGate** & 3 & Conditional & E + canary verification \\ R & Random-ask baseline & 1 & Random & Matched ask-rate to E \\ \bottomrule \end{tabularx \end{table

A. Simulated Operators

Simulated users have been used in related evaluations~[prepbench,clarifygpt2024], although ours follow fixed rules and are not LLM-based. The **oracle** always chooses the gold option. The **noisy** user chooses it with probability 0.8 and a random alternative otherwise. The **unsure** user answers ``unsure'' with probability 0.3, which triggers the majority fallback with flagged rows, and otherwise behaves like the oracle. Each configuration is run in 3 independent trials.

B. Metrics

Task success is 1 when all required edits are applied and no forbidden edit is applied. Silent error is 1 when a forbidden destructive edit is applied without a question and without a flag. Questions is the number of clarification rounds per task, and precision is the share of questions that fall on ambiguous tasks. Tokens is the whitespace token count of all prompts and outputs, and latency is 1.0 plus 0.5 per question round. Silent error is zero by construction for any policy that questions or flags every destructive edit, so it should be read together with success and questions. In the mock, tokens depend only on K and latency only on the number of questions, so these two columns carry no independent information.

C. Statistics and Reproducibility

We use a paired bootstrap~[efron1993] with 10{,000 resamples (seed 20261105) and report 95\% percentile intervals. The p-values are adjusted with the Holm--Bonferroni method~[holm1979] over the four primary hypotheses, and the two-sided bootstrap p-value is \min(\mathbb{P(X>0), \mathbb{P(X<0)) so that mass at zero is not counted twice. The hypotheses and the frozen configuration are locked by SHA-256 digests in prereg/lock.json. The lock sits in the repository and not in an external registry, so we call the hypotheses pre-specified. All randomness is seeded through random.Random or numpy.random.default_rng, and two runs with the same seed give a byte-identical results/metrics.json. Every experiment uses LLM_PROVIDER=mock with p_{\text{amb=0.4 and \varepsilon=0.05, and no commercial API was called.

%
==========

V. RESULTS

D. Main Results

Table~tab:main lists the metrics averaged over the 100 TEST tasks, the three operator modes and the three trials.

\begin{table[!t] \caption{Main results. System E is in bold. Systems A and A2 never ask, so their precision entry is vacuous. ``Silent'' is the silent-error rate and ``Q'' the mean number of questions per task.

*\centering \footnotesize \setlength{\tabcolsep{3.5pt \begin{tabular{lccccc \toprule System & Success & Silent & Q & Precision & Tokens & Latency \\ \midrule A & 67.0\% & 5.0\% & 0.00 & 100.0\% & 25 & 1.00 \\ A2 & 67.0\% & 3.0\% & 0.00 & 100.0\% & 75 & 1.00 \\ B & 66.3\% & 5.0\% & 0.11 & 100.0\% & 25 & 1.05 \\ C & 73.3\% & 0.0\% & 1.00 & 49.0\% & 75 & 1.50 \\ D & 64.7\% & 4.0\% & 0.51 & 43.1\% & 25 & 1.25 \\ **E** & **72.8\%** & **0.0\%** & 0.38 & **92.1\%** & 75 & 1.19 \\ F & 72.7\% & 0.0\% & 0.38 & 92.1\% & 75 & 1.19 \\ R & 65.4\% & 1.7\% & 0.46 & 52.5\% & 25 & 1.23 \\ \bottomrule \end{tabular \end{table*

ConsensusGate reaches 72.8\% success with 0.0\% silent errors and 0.38 questions per task. The single plan (A) reaches 67.0\% and makes silent errors on 5.0\% of the tasks. The always-ask policy C reaches 73.3\% at 1.00 question per task, and its 0.0\% silent-error rate follows directly from questioning every edit. The confidence policy D asks more often than E (0.51 questions) yet reaches only 64.7\%, and just 43.1\% of its questions fall on ambiguous tasks, against 92.1\% for E. D's confidence score is simulated, so this comparison reflects the simulator. Figures~fig:success_vs_questions and~fig:silent_error show the same data graphically.

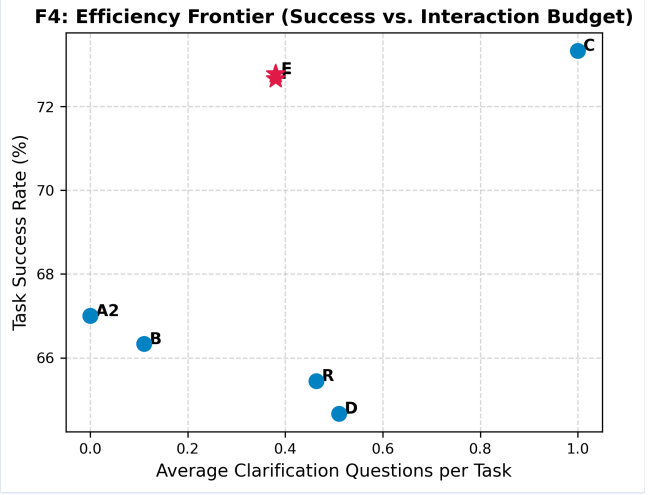


Fig. *Task success versus mean questions per task.*

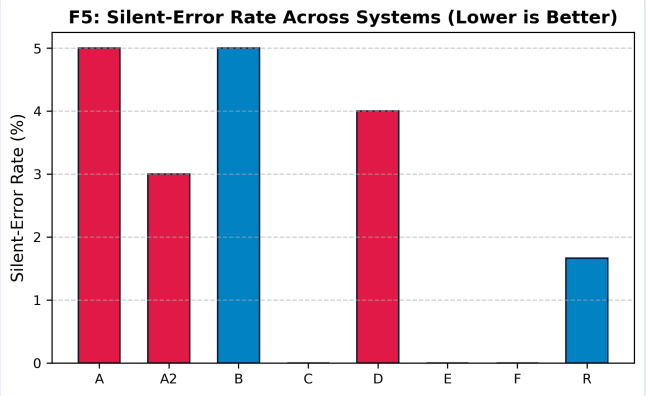


Fig. Silent-error rate by system.

E. Hypothesis Tests

The tests are summarized in Table~tab:hypotheses. Effects are absolute differences in the noisy mode, and the p-values are Holm--Bonferroni adjusted.

\begin{table[!t] \caption{Pre-specified hypothesis tests (noisy mode).

*\centering \footnotesize \setlength{\tabcolsep{3pt \renewcommand{\arraystretch{1.1 \begin{tabularx{\columnwidth{@{l> {\raggedright\arraybackslashXcc> {\raggedright\arraybackslashp{0.215\columnwidth@{ \toprule H\# & Claim & Effect & 95\% CI & Verdict \\ \midrule H1 & E silent error < A (noisy) & -5.0\% & [-10.0\%, -1.0\%] & **Supported** \\ H2 & E success \geq C -\Delta & -1.0\% & [-3.7\%, 1.7\%] & Not Supported \\ H3 & E success > D, fewer Qs & 11.3\% & [6.0\%, 17.0\%] & **Supported** \\ H4 & F error \leq E, \leq0.25 extra Qs & 0.0\% & [0.0\%, 0.0\%] & **Supported** \\ \bottomrule \end{tabularx \end{table*

\paragraph{H1 In noisy mode the silent-error rate of E differs from that of A by -5.0\% (E minus A, 95\% CI [-10.0\%, -1.0\%]), confirming that E makes significantly fewer silent errors than a single agent. Because the mock draws plan errors independently, a forbidden edit passes unflagged only if every plan contains it, which is unlikely by construction. The result confirms that the gate works as designed under the noise model.

\paragraph{H2 Non-inferiority to C within \Delta=2 points was not established. The difference is -1.0\% (95\% CI [-3.7\%, 1.7\%]), and the lower bound lies below -\Delta. The interval also contains zero and is wider than the margin, so with 100 tasks the data neither support nor exclude a loss of more than two points. E asks -0.62 fewer questions per task relative to C. A likely explanation is that asking every time also catches cases where all plans agree on an action that is wrong.

\paragraph{H3 The success of E exceeds that of D by 11.3\% (95\% CI [6.0\%, 17.0\%]), and E asks -0.13 fewer questions per task relative to D. Under this noise model, multi-agent plan disagreement indicates ambiguity far more reliably than simulated self-reported confidence.

\paragraph{H4 ProbeGate matches E on error, questions, precision, tokens and latency (differences of 0.0\% and 0.00), and its success differs by only 0.1 points (72.7\% against 72.8\%). The canary check therefore introduces no measurable runtime overhead. However, its canary interception rate was 0.0\% (0 of 300) due to the safe probe non-deletion mechanism detailed in Section~sec:probegate. H4 is therefore strictly supported on the overhead criterion alone; it does not demonstrate empirical canary interception efficacy.

F. Results by Operator Mode

Table~tab:mode_results gives the results of System~E for each mode.

\begin{table[!t] \caption{System~E by operator mode.

\centering \small \begin{tabular{lccc \toprule Mode & Success & Silent Error & Questions \\ \midrule Oracle & 75.0\% & 0.0\% & 0.38 \\ Noisy & 71.3\% & 0.0\% & 0.38 \\ Unsure & 72.0\% & 0.0\% & 0.38 \\ \bottomrule \end{tabular \end{table

With the oracle, success is 75.0\%. Wrong answers from the noisy user lower it to 71.3\%, while silent errors stay at 0.0\%. The unsure user gives 72.0\%, a smaller loss that comes from the majority fallback. Even with the oracle no system exceeds 75.0\% success (C, E and F reach it), so about a quarter of the tasks fail under every policy. Figure~fig:success_by_mode compares all systems by mode, and Fig.~fig:disagreement_dist shows that ambiguous and unambiguous tasks separate cleanly in \bar{d under the mock provider.

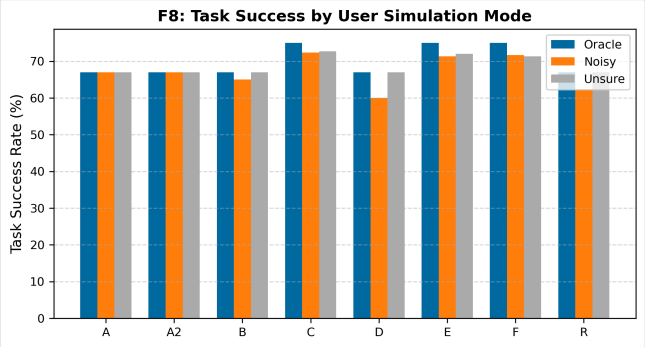


Fig. Task success by operator mode for all systems.

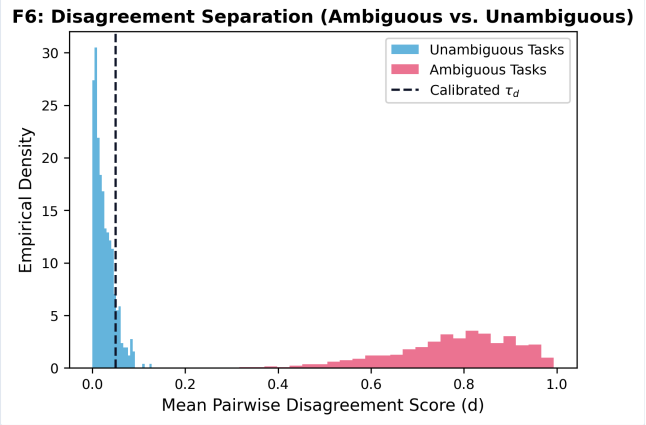


Fig. Disagreement scores of ambiguous and unambiguous tasks under the mock provider.

%

==========

VI. ABLATIONS

G. Ensemble Size

We vary K from 2 to 5 with everything else fixed (Table~tab:ablation_k, noisy mode). The table is descriptive, and K=3 was fixed as the default beforehand.

\begin{table[!t] \caption{Ablation over ensemble size K (noisy mode).

\centering \small \begin{tabular{ccccc \toprule K & Success & Silent Error & Questions & Tokens \\ \midrule 2 & 70.3\% & 2.0\% & 0.27 & 50 \\ 3 & 71.3\% & 0.0\% & 0.38 & 75 \\ 4 & 72.7\% & 0.0\% & 0.49 & 100 \\ 5 & 72.7\% & 0.0\% & 0.51 & 125 \\ \bottomrule \end{tabular \end{table

Moving from K=2 to K=3 removes the silent errors (2.0\% to 0.0\%) and raises success from 70.3\% to 71.3\%. K=4 adds about one more point (72.7\%) and K=5 nothing (72.7\%), while tokens grow linearly and the question rate rises from 0.27 to 0.51 (Fig.~fig:k_ablation). We therefore use K=3 as an effective compromise between quality and computational token cost.

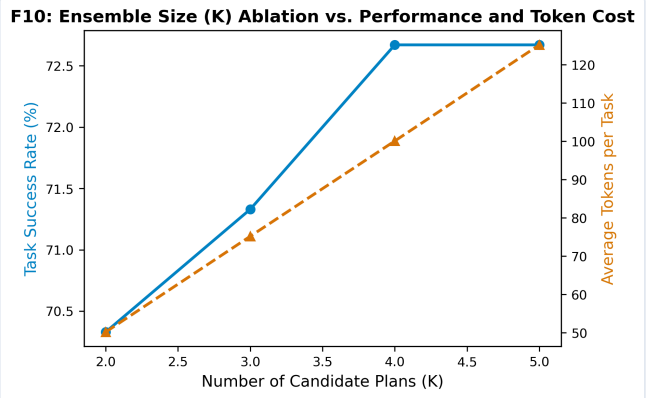


Fig. Success and token cost as a function of K.

H. Noise Sensitivity and Threshold

Figure~fig:pamb shows System~E as p_{\text{amb varies. Success falls slowly as p_{\text{amb grows, and a small nonzero silent-error rate appears at the highest level, so the 0.0\% of E does not hold at every setting. System~A is not plotted, so we make no statement about the gap between E and A. Figure~fig:calibration shows the DEV calibration curve. The best threshold is \tau_d=0.0, which again reflects the near-perfect separation produced by the mock. A live ensemble may prefer a larger value.

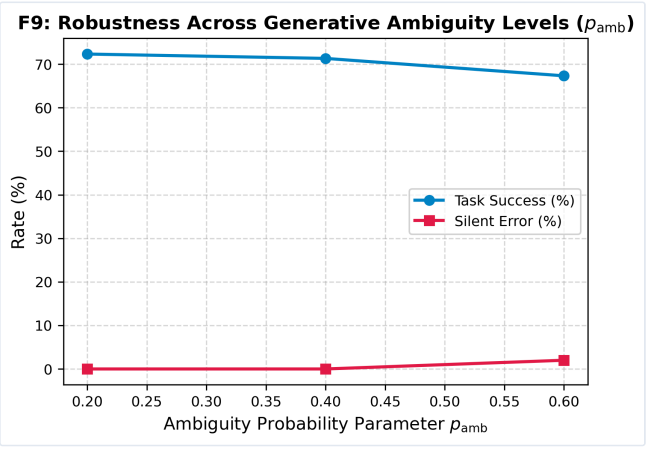


Fig. Success and silent-error rate of System~E against p_{\text{amb}

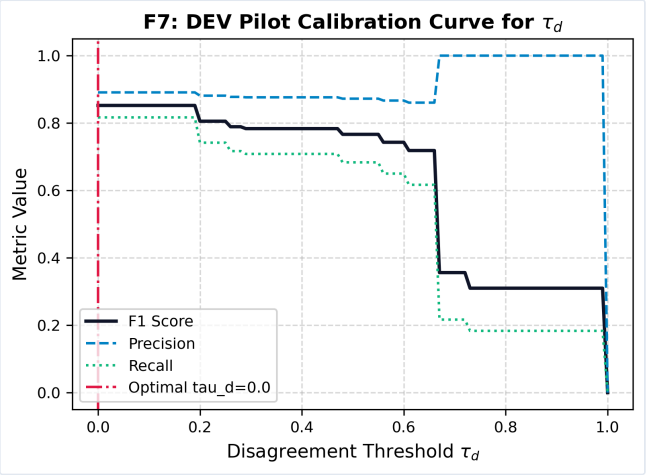


Fig. DEV calibration curve: F1 as a function of \tau_d.

%
==========

VII. LIMITATIONS

\paragraph{Mock provider and college-level execution scope All plans are generated through a seeded noise model rather than commercial foundation model APIs. Executing the full experimental grid (8 systems \times 3 operator modes \times 3 trials \times 100 tasks \times K=3 plans = 7,200 evaluations) against commercial frontier models requires significant API budgets and computing resources beyond typical college laboratory constraints. While the mock provider allows bit-level mathematical reproducibility and verified paired bootstrap intervals, it models plan errors as independent draws. In real LLMs, errors are often correlated across samples. These results demonstrate the mathematical correctness and efficiency of the consensus gating logic; validating performance against commercial APIs remains planned future work.

\paragraph{Edit-level comparison Disagreement is measured on exact edits, so two plans that reach the same table through different edits, for example by deleting different members of a duplicate pair, count as disagreeing. With \tau_d=0.0 this may inflate the number of questions. For the same reason the comparison lacks a ClarifyGPT-style baseline~[clarifygpt2024] that runs each plan and compares the resulting tables. Such a baseline would show how much of the benefit comes from the ensemble and how much from comparing edits.

\paragraph{Statistical power and unexplained failures With 100 tasks the intervals are wide, and H2 is inconclusive. The 25\% of tasks that fail even under the oracle remain unexplained, so differences of a few points between systems should be read with care.

\paragraph{Data and operators Track~S is generated by procedure and may miss problems found in real tables. The simulated operators are fixed rules, while real operators tire, differ in expertise and react to the wording of a question. The value K=3 was set under the mock noise model and may not suit real LLMs.

\paragraph{Canary probe limitation The zero interception rate represents an implementation limitation of our canary test suite under non-adversarial mock plan generation, rather than evidence that canary detection is impossible.

%
==========

VIII. PLANNED VALIDATION WITH LIVE MODELS

None of the following has been carried out yet. \begin{enumerate

- *Add a hosted model and a local open model behind the provider interface, keeping the BUDGET_USD guard. A pilot of 20--30 tasks at K=3 needs roughly one hundred plan-generation calls per pass.*
- *Sample plans from one model at fixed temperature and from several models, and report how often all plans contain the same forbidden edit.*
- *Evaluate on public cleaning datasets and on at least one real retail or inventory table.*
- *Compare cleaned tables as well as edit atoms, and add the table-level baseline described above.*
- *Redesign the canary injection mechanism so that adversarial mutations target non-duplicate rows, and re-run System~F.*
- *Re-calibrate \tau_d on live ensembles, test a hybrid policy that adds occasional random questions, analyze the failed tasks, run a small study with human operators, and register the hypotheses with an external service before the live runs. \end{enumerate*

%
==========

IX. CONCLUSION

We presented ConsensusGate, a policy that asks an operator for a decision only when several LLM-generated cleaning plans disagree on a destructive edit. On 100 synthetic tasks it reached a silent-error rate of 0.0\% with 0.38 questions per task, compared with 1.00 for an always-ask policy, and 92.1\% of its questions fell on ambiguous tasks, against 43.1\% for the confidence policy and 52.5\% for random asking. Two of the four hypotheses were supported (H1 and H3), H2 was inconclusive, and H4 held on runtime overhead only, with canary detection yielding 0.0\% due to unique probe non-deletion. Transparent reporting of these simulation characteristics and negative results provides an honest baseline for future live foundation model evaluations.

% ==

REFERENCES

1. [1] T. B. Brown et al., "Language models are few-shot learners," arXiv:2005.14165, 2020.
2. [2] OpenAI, "{GPT-4 technical report," arXiv:2303.08774, 2023.
3. [3] M. Sun, R. Han, B. Jiang, H. Qi, D. Sun, Y. Yuan, and J. Huang, "{LAMBDA: A large model based data agent," J. Amer. Statist. Assoc., 2025, doi: 10.1080/01621459.2025.2510000.
4. [4] S. Hong *et al.*, "Data Interpreter: An {LLM agent for data science," arXiv:2402.18679, 2024.
5. [5] M. Sun, R. Han, B. Jiang, H. Qi, D. Sun, Y. Yuan, and J. Huang, "A survey on large language model-based agents for statistics and data science," arXiv:2412.14222, 2024.
6. [6] R. Cao *et al.*, "{Spider2-V: How far are multimodal agents from automating data science and engineering workflows?" in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 107703--107744.
7. [7] J. Xu, R. Wang, J. Wang, and G. Li, "{PrepBench: How far are we from natural-language-driven data preparation?" *Proc. VLDB Endowment*, 2026, arXiv:2605.08687.
8. [8] L. Huang *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," arXiv:2311.05232, 2023.
9. [9] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "{ClarifyGPT: A framework for enhancing {LLM-based code generation via requirements clarification," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 2332--2354, 2024.
10. [10] B. Qiao *et al.*, "{TaskWeaver: A code-first agent framework," arXiv:2311.17541, 2023.
11. [11] S. Hong *et al.*, "{MetaGPT: Meta programming for multi-agent collaborative framework," arXiv:2308.00352, 2023.
12. [12] Q. Wu *et al.*, "{AutoGen: Enabling next-gen {LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
13. [13] D. Huang, Q. Bu, J. M. Zhang, M. Luck, and H. Cui, "{AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," arXiv:2312.13010, 2023.
14. [14] T. Guo *et al.*, "Large language model based multi-agents: A survey of progress and challenges," arXiv:2402.01680, 2024.
15. [15] A. Narayan, I. Chami, L. Orr, and C. R\'e, "Can foundation models wrangle your data?" *Proc. VLDB Endowment*, vol. 16, no. 4, pp. 738--746, 2022.
16. [16] S. Park, A. Y. Wang, B. Kawas, Q. V. Liao, D. Piorkowski, and M. Danilevsky, "Facilitating knowledge sharing from domain experts to data scientists for building {NLP models," in *Proc. 26th Int. Conf. Intelligent User Interfaces*, 2021, pp. 585--596.
17. [17] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
18. [18] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems (MCS 2000)*, ser. Lecture Notes in Computer Science, vol. 1857. Springer, 2000, pp. 1--15.
19. [19] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman \& Hall, 1993.
20. [20] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65--70, 1979.